

NeutronMP: Multi-Model Parallelism for Breaking Synchronization Barriers in Distributed GNN Training

Chunyang Li, Xin Ai, Yanfeng Zhang

School of Computer Science and Engineering, Northeastern University, Shenyang, China
20236521@stu.neu.edu.cn, aixin0@stumail.neu.edu.cn, zhangyf@mail.neu.edu.cn

Abstract—Distributed Graph Neural Network (GNN) training is often bottlenecked by iterative synchronization across workers. Besides gradient synchronization, GNNs require frequent cross-partition communication for message passing, making training highly sensitive to stragglers and resource fluctuations in shared cloud environments. Existing systems mainly reduce or hide synchronization costs through stale information reuse, but they still follow a single-global-model paradigm in which workers jointly optimize shared parameters. As a result, synchronization remains on the critical path of distributed training.

We present NeutronMP, a distributed GNN training system based on multi-model parallelism. Instead of jointly training one global model, NeutronMP decomposes training into multiple independently optimized submodels over graph partitions, thereby removing iterative cross-worker synchronization from the submodel training phase. After submodel training, NeutronMP performs a unified inference pass to generate cached intermediate embeddings and trains a lightweight stacking-based integration unit to fuse knowledge from different partitions. To make this paradigm practical, NeutronMP further introduces role-isolated execution, embedding caching, and topology-aware submodel pruning to improve resource utilization and reduce inference overhead. Experiments show that NeutronMP achieves an average 1.35 $\times$ end-to-end training speedup over existing distributed GNN training frameworks, while maintaining competitive accuracy.

Index Terms—Distributed GNNs, Asynchronous Training, Ensemble Learning, Cloud Computing

I. INTRODUCTION

Graph Neural Networks (GNNs) are increasingly used to analyze large-scale graph data in applications such as recommendation, social network analysis, and scientific computing [24]–[30], [35], [52]–[54]. Their training process is inherently data-intensive: each layer repeatedly accesses vertex features, traverses graph topology, and aggregates messages from neighboring vertices. As graph size exceeds the memory and compute capacity of a single machine, distributed training becomes necessary. A common solution partitions the graph across multiple workers and executes GNN training as a distributed dataflow over partitioned graph data [11], [12], [14], [31].

However, distributed GNN training remains highly sensitive to synchronization. Different from conventional distributed deep learning, GNN training requires not only parameter synchronization but also cross-partition message exchange

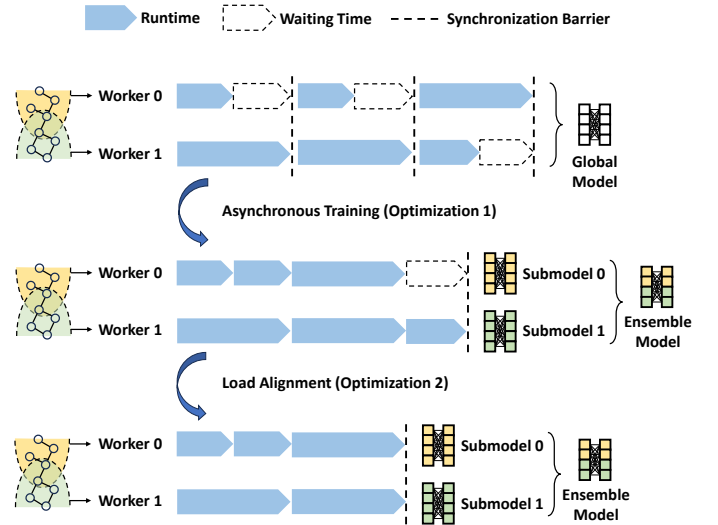


Fig. 1: Asynchronous training with time-based load alignment: Independent sub-models eliminate global synchronization barriers, allowing workers to execute varying epoch counts within aligned time windows.

for boundary vertices. These dependencies appear repeatedly across layers and iterations, placing communication and synchronization on the critical path. The problem becomes more severe in shared cloud environments, where graph partitions exhibit structural skew and worker performance fluctuates due to resource contention and network jitter. Once a slow worker appears, other workers wait at synchronization barriers, causing low resource utilization and unstable training throughput [17]–[19].

Existing systems [6], [13], [15], [16], [32], [36] mainly mitigate this problem by reducing the cost of cross-partition dependencies. A common strategy is to use historical embeddings or cached intermediate states when fresh remote embeddings are unavailable. This design reduces waiting time and allows part of the computation to proceed with stale information [33], [34], [46]. Nevertheless, it still follows the single-global-model paradigm: all workers jointly optimize the same model and need to maintain a consistent training process

over partitioned data. As a result, historical embeddings shift or relax synchronization, but do not remove iterative synchronization from the training loop.

This paper presents NeutronMP, a distributed GNN training system based on Multi-Model Parallelism. The key idea is to replace iterative joint optimization of a single global model with decoupled training of multiple partition-specialized submodels, followed by post-training integration. As shown in Figure 1, conventional distributed training repeatedly waits at synchronization barriers before updating a global model. NeutronMP instead lets each worker train an independent submodel on its local graph partition without iterative cross-worker synchronization. Moreover, NeutronMP adopts time-based load alignment: workers train within the same wall-clock window but may execute different numbers of epochs, which improves resource utilization under load imbalance and resource fluctuation.

After submodel training, NeutronMP constructs a unified inference pipeline to generate intermediate embeddings and trains a lightweight stacking-based integration unit to fuse knowledge from different graph partitions. This design shifts global integration out of the iterative training loop and turns it into a post-training stage over cached embeddings. To make this paradigm practical for large-scale graph data, NeutronMP further introduces three system techniques: role-isolated execution to separate compute-intensive submodel training from I/O-intensive embedding generation and integration, embedding caching to support efficient repeated access to intermediate representations, and topology-aware submodel pruning to reduce the inference overhead of activating many submodels.

Overall, NeutronMP makes the following contributions. First, it introduces Multi-Model Parallelism as a distributed GNN training paradigm that removes iterative cross-worker synchronization from the submodel training phase. Second, it designs a practical system workflow that combines decoupled submodel training, cached embedding generation, and stacking-based integration. Third, it develops system optimizations for resource isolation, embedding management, and topology-aware pruning. Experiments show that NeutronMP improves end-to-end training performance by 1.35 $\times$ on average and up to 12.85 $\times$ over existing distributed GNN training frameworks, while maintaining competitive accuracy under unstable training conditions.

II. BACKGROUND AND MOTIVATION

A. Distributed GNN Training Paradigms

Graph Neural Networks (GNNs) learn node representations through iterative message passing [20]–[22], [39], [50]. Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of vertices and $\mathcal{E}$ is the set of edges, the embedding $\mathbf{h}_v^{(l)}$ of vertex v at layer l is computed by aggregating information from its neighbors $\mathcal{N}(v)$ and applying a neural transformation:

$$\mathbf{a}_v^{(l)} = \text{AGGREGATE}^{(l)} \left(\{ \mathbf{h}_u^{(l-1)} \mid u \in \mathcal{N}(v) \} \right), \quad (1)$$

$$\mathbf{h}_v^{(l)} = \text{UPDATE}^{(l)} \left(\mathbf{a}_v^{(l)}, \mathbf{h}_v^{(l-1)}; \mathbf{W}^{(l)} \right), \quad (2)$$

where $\mathbf{W}^{(l)}$ denotes the trainable parameters. To handle industrial-scale graphs, distributed training partitions the graph across multiple workers. Existing approaches primarily rely on three parallelism strategies: *Graph Parallelism* (partitioning vertices/edges), *Tensor Parallelism* (partitioning feature dimensions), and *Pipelined Model Parallelism* (partitioning model layers) [23], [37], [41]–[44].

Despite their prevalence, these methods are constrained by the “single global model” architecture, where data partitioning is tightly coupled with shared parameters. Table I compares traditional strategies with our proposed Multi-Model Parallelism. While traditional methods incur frequent synchronization and communication, our strategy achieves complete decoupling during the training phase, eliminating both overheads until the final integration stage [5], [6].

B. Motivation

In practical training scenarios, the performance of distributed GNN training is constrained not only by communication overhead but also significantly by load imbalance and resource fluctuations.

a) Load Imbalance Challenges.: Due to the inherent unstructured nature of graph data [47], [49], [51], achieving perfectly balanced computational loads across subgraphs is difficult even with advanced partitioning algorithms. Certain nodes must process significantly more edges and neighbor aggregations, creating stragglers. In synchronous training modes, all workers must wait for the slowest node to complete gradient computation before parameter updates can proceed, leading to resource idleness on faster nodes and bounding overall efficiency by the slowest worker [45]–[48].

b) Resource Fluctuations in Cloud Environments.: Resource fluctuations are ubiquitous in real-world cloud training environments, often caused by contention on shared physical hosts or network congestion. Specifically, computational fluctuations arise when neighboring virtual machines on multi-tenant clusters demand high resources, causing CPU/GPU performance degradation (e.g., in burstable instances). Network fluctuations stem from datacenter switch overloads and cross-worker bandwidth contention. These fluctuations significantly impact computing power (FLOPS) and network bandwidth, severely degrading distributed GNN training performance [38], [40], [46], [48].

C. Potential of Ensemble Learning in GNNs

Ensemble learning enhances model robustness by combining multiple base learners. While strategies include Bagging and Boosting, this work focuses on Stacking, which employs a meta-learner to fuse outputs from independently trained base models. This two-stage architecture inherently supports computational decoupling, making it ideal for distributed parallelism. Specifically, Stacking allows base models to specialize in distinct graph partitions, aggregating localized knowledge while restoring global representational capacity.

Although prior studies have applied ensemble methods to GNNs [1]–[4], they primarily target accuracy improvements

Strategy	Partition Object	Sync Freq. (per Epoch)	Comm. Volume (per Epoch)
Data Parallelism	Graph Structure	L (Layer-wise Communication)	$O(L \cdot \partial V \cdot H)$
Tensor Parallelism	Feature Dim	$2L$ (Layer-wise Gather/Split)	$O(L \cdot N \cdot H)$
Pipeline Parallelism	Model Layers	L (Inter-layer Embedding)	$O(N \cdot H)$
Multi-Model Parallelism	Graph Structure + Global Model	0	0 (Training); $O(M_{param})$ (Final)

TABLE I: Theoretical comparison of distributed GNN parallelism strategies. Here, L is the number of layers, N the number of vertices, H the hidden dimension, K the number of workers, $|\partial V|$ the number of boundary vertices, and M_{param} the total model parameter size.

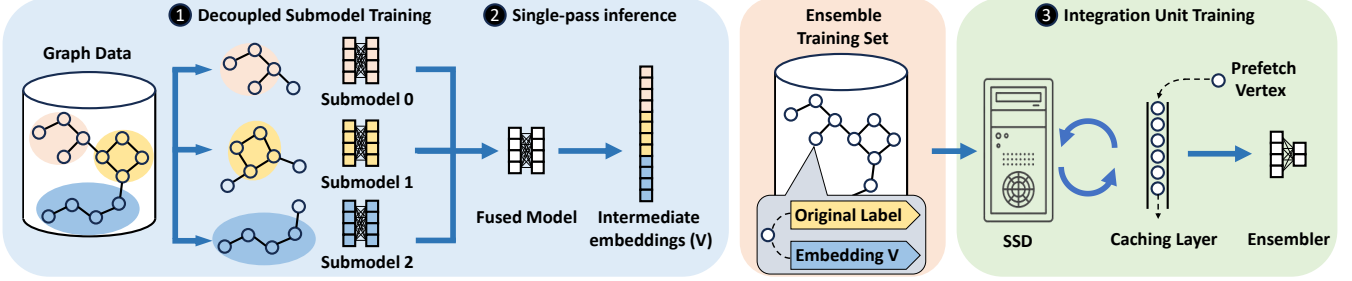


Fig. 2: The three-phase training pipeline of GNN Multi-Model Parallelism: (1) decoupled submodel training, (2) ensemble training set generation through single-pass inference, and (3) integration unit training.

via sequential or voting mechanisms, overlooking the inherent efficiency of parallel-friendly ensemble structures. In contrast, we demonstrate that leveraging Stacking for parallel GNN training fundamentally resolves communication and synchronization bottlenecks, yielding a highly efficient, robust, and flexible framework.

III. GNN MULTI-MODEL PARALLELISM

This section presents NEUTRONMP, a distributed GNN training framework grounded in *Multi-Model Parallelism* (MP). The core principle is that training-phase decoupling need not compromise inference-phase unity: independently trained submodels on graph partitions are seamlessly integrated into a single cohesive model compatible with standard deployment pipelines. This design eliminates communication and synchronization bottlenecks during training while preserving full model expressivity.

A. Parallelism Workflow

The NEUTRONMP workflow operates in three logically distinct phases. First, **decoupled submodel training**. The input graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is partitioned into K independent subgraphs $\{\mathcal{G}_k = (\mathcal{V}_k, \mathcal{E}_k)\}_{k=1}^K$. In general scenarios, we employ neighbor node replication to enrich local context without introducing cross-device communication during the training phase. Each worker k independently trains a lightweight submodel $\mathcal{M}_k$ on $\mathcal{G}_k$, which functions as a self-contained and complete GNN. Critically, no inter-worker communication or synchronization occurs during this phase; workers train asynchronously until convergence based on a fixed wall-clock duration, thereby eliminating resource idleness caused by stragglers.

Second, **ensemble training set generation**. Upon completion of submodel training, the system constructs a unified

model $\mathcal{M}_{unified}$ by concatenating the neural transformation parameters of all submodels, i.e., $f_{nn}^{unified} = \bigoplus_{k=1}^K f_{nn}^{(k)}$, while retaining a single shared graph aggregation operator f_{agg} . $\mathcal{M}_{unified}$ executes a single-pass inference: for each node v in original training set, it first computes the aggregated neighborhood representation $\mathbf{z}_v = f_{agg}(\{\mathbf{h}_u \mid u \in \mathcal{N}(v)\})$, and then parallelly applies all K neural transformation modules to obtain the concatenated embedding vector $\mathbf{e}_v = [\mathbf{e}_v^{(1)}; \dots; \mathbf{e}_v^{(K)}]$, where $\mathbf{e}_v^{(k)} = f_{nn}^{(k)}(\mathbf{z}_v)$. The generated embeddings $\{\mathbf{e}_v\}_{v \in \mathcal{V}}$ are cached to persistent storage for the subsequent phase.

Third, **integration unit training**. A lightweight integration unit $\mathcal{I}$ is trained independently on the cached embeddings $\{\mathbf{e}_v\}$ with original labels. This unit learns to fuse local knowledge from different subgraphs and recover long-range structural dependencies truncated by the initial graph partitioning. Since this phase accesses only cached embeddings, it requires no graph traversal or remote communication, allowing for efficient multi-epoch training. The final inference model is defined as $\mathcal{M}_{final} = \mathcal{I} \circ \mathcal{M}_{unified}$, which maintains compatibility with standard GNN deployment engines.

B. Theoretical Analysis

We analyze two properties: **Training loss matching** – the ensemble can achieve training loss as low as a global GNN; **Generalization** – the ensemble model class has a controlled test error bound.

For the generalization analysis only, we assume: graph aggregation is parameter-free and non-expansive (e.g., mean pooling); activations are 1-Lipschitz; biases zero; weights spectrally normalized; loss ℓ is L_ℓ -Lipschitz and bounded in $[0, 1]$. The training loss matching result relies only on over-parameterization of the **integrator MLP** (Phase 3), not on GNN optimization; we assume that submodels and the

global GNN can be trained to sufficiently low loss (as is common in practice).

Part I: Training Loss Matching. We perform L -hop neighbor replication when constructing subgraphs $\mathcal{G}_k$. This ensures that for every node $v \in \mathcal{V}_k$, its L -hop neighborhood is identical to that in the full graph. Hence any L -layer GNN receives the same set of neighbor features for v whether running on $\mathcal{G}_k$ or on $\mathcal{G}$.

Let f_{global} be a global GNN (depth L , hidden width W) trained on the full graph. For each partition k , let h_k be a GNN of the same architecture trained independently on $(\mathcal{G}_k, \mathcal{V}_k)$. We assume (and can achieve in practice with sufficient width and iterations) that both have small training loss:

$$\begin{aligned}\widehat{L}_{\text{global}} &= \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \ell(f_{\text{global}}(v), y_v) \leq \epsilon, \\ \widehat{L}_k &= \frac{1}{|\mathcal{V}_k|} \sum_{v \in \mathcal{V}_k} \ell(h_k(v), y_v) \leq \epsilon.\end{aligned}$$

Define the ensemble embedding

$$\mathbf{e}_v = [h_1(\mathbf{x}_v); \dots; h_K(\mathbf{x}_v)],$$

where $\mathbf{x}_v$ denotes the raw features of v (the aggregation is performed inside each h_k). Consider the routing function $\phi(\mathbf{e}_v) = h_{k(v)}(v)$ where $k(v)$ is the partition containing v . Since the training set is finite and the ensemble embeddings $\{\mathbf{e}_v\}$ are distinct for nodes with different labels (a mild separability condition naturally satisfied in high-dimensional spaces), there exists a two-layer MLP $\mathcal{I}^*$ that exactly interpolates these pairs (a standard finite-sample universal approximation). Then the composed model $f^* = \mathcal{I}^* \circ \mathbf{e}$ satisfies for all v :

$$\ell(f^*(v), y_v) = \ell(h_{k(v)}(v), y_v).$$

Therefore, the average training loss of f^* is

$$\widehat{L}(f^*) = \frac{1}{|\mathcal{V}|} \sum_{k=1}^K \sum_{v \in \mathcal{V}_k} \ell(h_k(v), y_v) = \sum_{k=1}^K \frac{|\mathcal{V}_k|}{|\mathcal{V}|} \widehat{L}_k \leq \epsilon.$$

Now we train the integrator $\mathcal{I}_\phi$ via gradient descent on the cached dataset $\{(\mathbf{e}_v, y_v)\}$, with all h_k frozen. The empirical loss $\widehat{L}_{\text{ens}}(\phi) = \frac{1}{|\mathcal{V}|} \sum_v \ell(\mathcal{I}_\phi(\mathbf{e}_v), y_v)$ is minimized over ϕ . By the theory of over-parameterized two-layer MLPs [1,2], for sufficiently large width $W_{\mathcal{I}}$ (polynomial in $|\mathcal{V}|$ and $1/\delta$), gradient descent converges to a global minimum with training loss at most that of any feasible point. Since $\mathcal{I}^*$ is feasible,

$$\min_{\phi} \widehat{L}_{\text{ens}}(\phi) \leq \widehat{L}_{\text{ens}}(\mathcal{I}^*) = \widehat{L}(f^*) \leq \epsilon.$$

Theorem 1 (Training Loss Matching). Under the above conditions, for any $\epsilon > 0$, there exists a width $W_{\mathcal{I}}$ such that the integrator trained by gradient descent yields an ensemble f_{ens} with

$$\frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \ell(f_{\text{ens}}(v), y_v) \leq \epsilon.$$

Thus, the decoupled integrator introduces no expressivity bottleneck and achieves training loss as low as that of a global

GNN. Note that the assumption $\widehat{L}_k \leq \epsilon$ is not a circular prerequisite; rather, since h_k shares the same architecture as f_{global} and is trained on a smaller node subset ($|\mathcal{V}_k| \leq |\mathcal{V}|$), standard over-parameterization theory guarantees that if the global GNN can reach ϵ , the submodels can reach it at least as easily. In practice, the integrator may learn non-linear combinations that further reduce loss.

Part II: Generalization Bound.

We now analyze the test performance of the ensemble model class $\mathcal{F} = \{\mathbf{x} \mapsto \mathcal{I}(\mathbf{e}(\mathbf{x}))\}$, where $\mathbf{e}(\mathbf{x}) = [h_1(\mathbf{x}); \dots; h_K(\mathbf{x})]$ are frozen submodels and $\mathcal{I}$ is an MLP with Lipschitz constant Λ (enforced by spectral normalization). Assume: $\|\mathbf{x}_v\|_2 \leq B$; each h_k has Lipschitz constant $\leq M_k$ (with respect to ℓ_2) and output dimension d_h ; loss ℓ is L_ℓ -Lipschitz and bounded in $[0, 1]$. Let C be the number of classes.

Denote by $\mathfrak{R}_n(\cdot)$ the empirical Rademacher complexity and by $\mathfrak{R}_n^{\text{vec}}(\cdot)$ its vector-valued extension. We have the following standard lemmas (proofs are omitted for brevity; see e.g., [3,4]).

Lemma 2 (Loss Contraction).

$$\mathfrak{R}_n(\mathcal{L}) \leq \sqrt{2} L_\ell \mathfrak{R}_n^{\text{vec}}(\mathcal{F}),$$

where $\mathcal{L} = \{(\mathbf{x}, y) \mapsto \ell(f(\mathbf{x}), y) : f \in \mathcal{F}\}$.

Lemma 3 (Decomposition).

$$\mathfrak{R}_n^{\text{vec}}(\mathcal{F}) \leq \sqrt{2} C \Lambda \sum_{k=1}^K \mathfrak{R}_n^{\text{vec}}(\mathcal{H}_k),$$

where $\mathcal{H}_k$ is the class of submodel h_k .

Lemma 4 (Submodel Complexity). Let $\mathcal{H}_k$ be the function class of GNNs with bounded weight norms (ensuring a finite Lipschitz constant M_k under our non-expansive aggregation assumption). For any $h_k \in \mathcal{H}_k$ with $h_k(0) = 0$,

$$\mathfrak{R}_n^{\text{vec}}(\mathcal{H}_k) \leq \frac{c B M_k \sqrt{d_h}}{\sqrt{n}},$$

where c is an absolute constant.

Theorem 2 (Generalization). For any $\delta \in (0, 1)$, with probability at least $1 - \delta$ over an i.i.d. mini-batch S of size n drawn from the node distribution,

$$R(f) \leq \widehat{R}_S(f) + \frac{c_0 L_\ell C \Lambda B}{\sqrt{n}} \sum_{k=1}^K M_k \sqrt{d_h} + \sqrt{\frac{\log(1/\delta)}{2n}}, \quad \forall f \in \mathcal{F},$$

where $R(f) = \mathbb{E}[\ell(f(\mathbf{x}), y)]$ is the true risk and $\widehat{R}_S(f)$ the empirical risk on S .

Proof. Uniform convergence gives $R(f) - \widehat{R}_S(f) \leq 2\mathfrak{R}_n(\mathcal{L}) + \sqrt{\log(1/\delta)/(2n)}$. Substituting Lemmas 2–4 yields the bound. *Comparison.* A monolithic GNN ($K = 1$, $\Lambda = 1$, $d_h = C$) has penalty $\frac{c_1 L_\ell C B}{\sqrt{n}} M_{\text{full}}$. Our bound replaces $M_{\text{full}} C$ with $\Lambda \sum_k M_k \sqrt{d_h}$. The dependence on K is linear, but each term can be much smaller if subgraphs are locally simple (small M_k, d_h). Hence parallelization does not cause a statistical catastrophe.

Summary.

- **Training loss matching (Theorem 1):** With a sufficiently wide integrator, gradient descent can reach training loss

as low as that of a global GNN, given that submodels already have low loss. This shows no optimization barrier.

- **Generalization (Theorem 2):** The ensemble model class has a generalization bound that scales linearly with K and can be tighter under local simplicity.

C. GNN Ensemble Friendliness Analysis

The efficiency of Multi-Model Parallelism in the GNN domain stems from the unique computational structure of GNNs, which distinguishes them from conventional deep learning architectures. The graph aggregation operator in a GNN involves two sequential stages: (i) *graph sampling*, encompassing neighbor indexing, computational subgraph construction, and feature loading; and (ii) *message passing*, implemented via sparse matrix multiplication (SpMM) over the sampled edges. Among these stages, graph sampling constitutes the primary performance bottleneck: it requires extensive random memory access to adjacency structures and incurs substantial CPU-to-GPU data transfer overhead, whereas SpMM and dense matrix multiplication are regular, bandwidth-efficient GPU kernel computations with comparatively modest latency.

This asymmetry creates a natural opportunity for ensemble acceleration. When K submodels are co-located within a unified model, fusing the graph aggregation operator allows the expensive graph sampling stage to be shared across all submodels. At the first layer, all submodels share identical input features, so the SpMM operates on the original D -dimensional tensor, and the aggregated representations are distributed to K parallel transformation branches. At subsequent layers, the SpMM operates on the concatenated KD -dimensional tensor. Mathematically, this fused SpMM is equivalent to multiplication by a block-diagonal matrix in which only the K diagonal blocks of size $D \times D$ are nonzero; hence the effective parameter count and per-submodel arithmetic remain unchanged. As the number of submodels increases, the additional cost is confined to the linear growth of SpMM computation in deeper layers, while the expensive sampling and data preparation pipeline is fully amortized.

Formally, consider an L -layer GNN. Let T_{agg} denote the latency of graph sampling at a single layer (neighbor indexing, subgraph construction, and feature transfer), and T_{mm} denote the latency of SpMM. Under the conventional ensemble paradigm, each of the K submodels executes an independent pipeline at every layer, and the total cost of graph aggregation operators is $KL(T_{\text{agg}} + T_{\text{mm}})$. Under Multi-Model Parallelism, graph sampling at each layer is performed once rather than K times, contributing $L \cdot T_{\text{agg}}$. The matrix computation cost is $T_{\text{mm}} + (L-1) \cdot K \cdot T_{\text{mm}}$: the first-layer SpMM operates on the shared D -dimensional input at unit cost, while each subsequent layer operates on the concatenated KD -dimensional hidden state at cost $K \cdot T_{\text{mm}}$. The total cost thus becomes $L \cdot T_{\text{agg}} + T_{\text{mm}}(1 + K(L-1))$. Since $T_{\text{agg}} \gg T_{\text{mm}}$ in large-scale training, the reduction of sampling from KL invocations to L dominates the overall speedup, and the wall-clock integration overhead grows sublinearly with K .

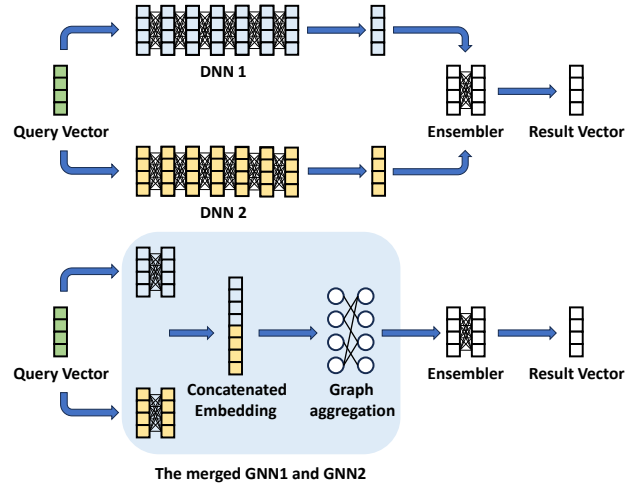


Fig. 3: Comparison of ensemble learning efficiency: Unlike traditional DNNs where sub-models require independent, non-overlapping computations, GNNs leverage shared graph propagation operators during inference, yielding an inherent efficiency advantage.

D. Comparison with Other Ensemble Methods

While various ensemble strategies utilize multiple submodels, the Stacking-based paradigm adopted in NEUTRONMP differs fundamentally from other approaches in design philosophy and system behavior.

a) Stacking vs. Simple Averaging.: Simple ensemble methods (e.g., Bagging with voting) typically fuse submodel outputs via fixed weights, such as uniform averaging. While computationally inexpensive, this approach limits expressiveness as it cannot learn complex interactions between submodel predictions. In contrast, our Stacking-based approach employs a learnable meta-learner (the integration unit) that adaptively weights and fuses submodel outputs. This enables the system to capture complementary strengths among submodels trained on different graph partitions, yielding superior accuracy, particularly when partitions exhibit heterogeneous structural properties.

b) Stacking vs. Mixture-of-Experts (MoE).: While MoE architectures also leverage multiple submodels, their design philosophy diverges from our Stacking-based approach. MoE typically employs dynamic gating networks to route inputs to sparse subsets of experts, prioritizing computational efficiency. Although some MoE variants can utilize local gating to reduce communication, the dynamic routing mechanism still introduces complexity in load balancing and synchronization in distributed settings. More fundamentally, the two paradigms differ in context utilization: MoE’s sparse activation selects a subset of experts per sample, which may limit the full exploitation of global semantic contexts across all partitions. In contrast, NeutronMP adopts a dense activation strategy where all submodels contribute. While computationally heavier, our shared graph aggregation amortizes this cost. Crucially, this “post-hoc integration” ensures access to the complete semantic

landscape, explicitly recovering cross-partition dependencies that might be under-utilized by dynamically selected local experts. This offers more stable inference latency and robustness for partitioned graph training.

E. System Challenges

Despite the architectural advantages of Multi-Model Parallelism, deploying it in large-scale distributed systems presents critical challenges that must be addressed. We identify two primary problems below; specific system-level optimizations are detailed in Section IV.

a) *Coupling of I/O and Compute Tasks.*: GNN Multi-Model Parallelism tasks exhibit a distinct dichotomy between I/O-intensive and compute-intensive workloads. The Master process handles I/O-intensive tasks, such as embedding cache management and integration unit training, while Worker processes are dedicated to compute-intensive submodel training. Traditional mixed-deployment strategies often result in GPU idleness during I/O phases or CPU bottlenecks during aggregation. Physically segregating these tasks is necessary to eliminate resource contention, yet it requires careful system design to manage data handoff between roles efficiently.

b) *Lack of Computational Sparsity.*: Unlike MoE’s sparse activation, Ensemble Parallelism inherently activates all K submodels per node during unified inference, incurring $\mathcal{O}(K)$ computational overhead in the neural transformation phase. While the shared aggregation mechanism amortizes the dominant cost, the linear growth in neural transformation cost can still become a bottleneck under device-constrained conditions. Introducing controlled sparsity without complex gating networks remains an open challenge to further reduce inference costs while maintaining model expressiveness.

IV. THE NEUTRONMP

This section details the key system optimization strategies employed by NEUTRONMP to bridge the gap between the theoretical Multi-Model Parallelism paradigm and practical large-scale deployment. Addressing prevalent challenges in distributed training such as resource contention, computational redundancy, and memory bottlenecks, we design three core optimization mechanisms: an I/O-compute decoupled execution workflow, topology-aware submodel pruning, and OS-guided memory-mapped embedding management. Notably, during the submodel training phase, each node optimizes only its local parameters θ_k ; upon completion, these model parameters (rather than gradients) are communicated and aggregated to construct the unified model.

A. I/O-Compute Decoupled Execution Workflow

To mitigate resource contention under heterogeneous workloads, NEUTRONMP adopts a role isolation strategy, explicitly categorizing system tasks into I/O-intensive and compute-intensive types, which are deployed on dedicated nodes respectively. I/O-intensive nodes are primarily responsible for global graph loading, METIS partitioning, unified model construction, single-pass inference for embedding caching,

and the training of the integration unit; their performance bottlenecks lie mainly in memory bandwidth and storage read/write throughput. Conversely, compute-intensive nodes focus exclusively on asynchronous submodel training on their respective subgraphs, being bound entirely by GPU computational capacity. This physical segregation not only avoids GPU idleness during I/O waiting phases or CPU bottlenecks during aggregation stages inherent in traditional mixed-deployment strategies but also enables targeted hardware optimization: I/O nodes can be configured with large-capacity RAM and high-speed NVMe storage to enhance throughput, while compute nodes maximize GPU density. Regarding data interaction, the system flexibly selects between a shared high-performance file system or high-speed point-to-point communication channels depending on the cluster environment. This ensures that upon completion of submodel training, each node only needs to transmit its converged local model parameters θ_k to the I/O nodes for aggregation, after which resources can be released immediately. This achieves efficient decoupling during the training phase and maximizes resource utilization.

B. Topology-Aware Submodel Pruning

To address the computational overhead incurred by activating all K submodels during the inference phase in Ensemble Parallelism, NEUTRONMP proposes a static, topology-aware submodel pruning strategy. Unlike Mixture-of-Experts (MoE) systems that rely on dynamic gating networks, and to avoid synchronization overhead from runtime routing and interference with integration unit training caused by variable input dimensions, we leverage global graph topology to pre-determine a fixed sparse activation pattern. The core of this strategy lies in constructing a meta-graph structure that reflects inter-partition dependencies and designing a hierarchical expert selection mechanism based on it.

First, the system maps the K partitions of the original graph $\mathcal{G}$ into a weighted undirected meta-graph $\mathcal{G}_p = (\mathcal{V}_p, \mathcal{E}_p)$, termed the *partition graph*. Here, each node $v_p^{(i)} \in \mathcal{V}_p$ represents a submodel partition $\mathcal{G}_i$, and the weight w_{ij} of an edge $e_{ij} \in \mathcal{E}_p$ is defined as the number of cross-partition edges in the original graph connecting partitions $\mathcal{G}_i$ and $\mathcal{G}_j$:

$$w_{ij} = |\{(u, v) \in \mathcal{E} \mid u \in \mathcal{V}_i, v \in \mathcal{V}_j\}|. \quad (3)$$

This structure quantifies the intensity of information flow between different submodels; a larger weight implies tighter topological connectivity between two partitions, indicating that their corresponding submodels require mutual information supplementation more critically when capturing local structures. Based on the partition graph $\mathcal{G}_p$, we perform dual analysis to extract key topological features:

- 1) **Hotspot Expert Query**: We run the PageRank algorithm on $\mathcal{G}_p$ to compute the centrality score of each partition node, designating the top h partitions with the highest scores as “Hotspot Experts” ($\mathcal{H}$). These experts are capable of providing global contextual information spanning the entire graph, thereby compensating for long-range dependency loss caused by hard partitioning.

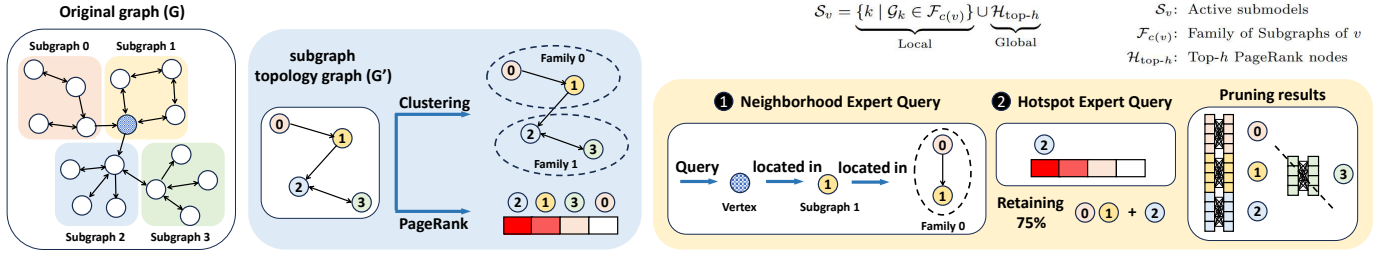


Fig. 4: Dual Clustering on Original Graph and Topology-Aware Subgraph Pruning.

2) **Neighborhood Expert Query:** We employ the METIS algorithm to perform secondary clustering on the partition graph $\mathcal{G}_p$, dividing the K partitions into C disjoint “Families” $\{\mathcal{F}_1, \dots, \mathcal{F}_C\}$. The objective is to maximize the sum of edge weights within families while minimizing cut edges between families, ensuring that submodels which are spatially proximate and topologically highly correlated are grouped into the same family.

Consequently, for any node v belonging to family $\mathcal{F}_c$, the set of active submodels $\mathcal{S}_v$ during inference consists of two components:

$$\mathcal{S}_v = \{k \mid \mathcal{G}_k \in \mathcal{F}_c\} \cup \mathcal{H}. \quad (4)$$

Specifically, node v activates all “Local Experts” within its corresponding family to capture dense local dependencies, while simultaneously activating the globally selected h “Hotspot Experts” to acquire a global perspective. This strategy significantly reduces the number of active submodels per node from K to $|\mathcal{F}_c| + h$ (typically much smaller than K). Since the pruning pattern is static and pre-computed based on topology, no additional routing computation is required during inference. Furthermore, each family can independently train a dedicated lightweight integration unit $\mathcal{I}_c$, further enhancing the system’s scalability and inference efficiency.

C. Implementation Details

To further address challenges in practical deployment, NEUTRONMP implements two critical optimizations. The first is the *time-based termination strategy*, designed to resolve load imbalance caused by structural skewness in graph partitions (e.g., power-law distributions). Since decoupled training eliminates synchronization barriers, forcing all nodes to complete an identical number of epochs would result in resource wastage for faster nodes. Therefore, the system adopts a fixed wall-clock duration as the termination condition, ensuring that all computational resources remain fully utilized until the global deadline. Empirical evidence demonstrates that, benefiting from the robustness of ensemble learning to minor variations in training duration, this strategy significantly improves GPU utilization while having negligible impact on model convergence accuracy. The second optimization is *OS-guided memory-mapped embedding management*, addressing the challenge where massive intermediate embeddings ($\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times (K \cdot d)}$) exceed physical memory. The system eschews complex multi-tier software caches in favor of leveraging OS-level virtual

memory management combined with application-guided paging techniques: embedding data is persisted as raw binary files and loaded via `mmap`, enabling zero-copy access. Simultaneously, a lightweight page manager tracks access patterns and issues `madvise` hints to the kernel, proactively retaining “hot” pages while releasing “cold” ones. This mechanism effectively decouples the working set size from the total graph scale, making it feasible to train integration units for ultra-large-scale graphs on commodity hardware.

V. EVALUATION

In this section, we comprehensively evaluate the NeutronMP system through a series of extensive experiments. The experimental design aims to address the following four core research questions:

- 1) **Efficiency:** Compared with mainstream distributed GNN training frameworks, can NeutronMP significantly reduce end-to-end training latency?
- 2) **Robustness:** Can the system maintain stable performance under optimization scenarios such as topology-aware sub-model pruning?
- 3) **Scalability:** In simulated ultra-large-scale cluster environments, can the system’s integration strategy maintain model accuracy consistency while scaling up?
- 4) **Effectiveness of Optimizations:** Through ablation studies, we quantify the specific contributions of various system optimization strategies and the associated system overhead.

A. Experimental Setup

a) *Hardware Environment:* Distributed experiments are deployed on an Alibaba Cloud ECS cluster consisting of 4 gn7i-c16g1.4xlarge compute nodes. Each node is equipped with 16 vCPUs, 60 GiB of memory, and one NVIDIA A10 GPU with 24 GB of VRAM. Additionally, some comparative experiments are conducted in a local multi-GPU single-node environment. This server features dual Intel Xeon Silver 4316 CPUs, 503 GB of system memory, and three NVIDIA RTX A6000 GPUs, each with 48 GB of VRAM. To support NeutronMP’s efficient embedding persistence mechanism, the server is further configured with a 3.6 TB NVMe SSD as high-speed storage backend.

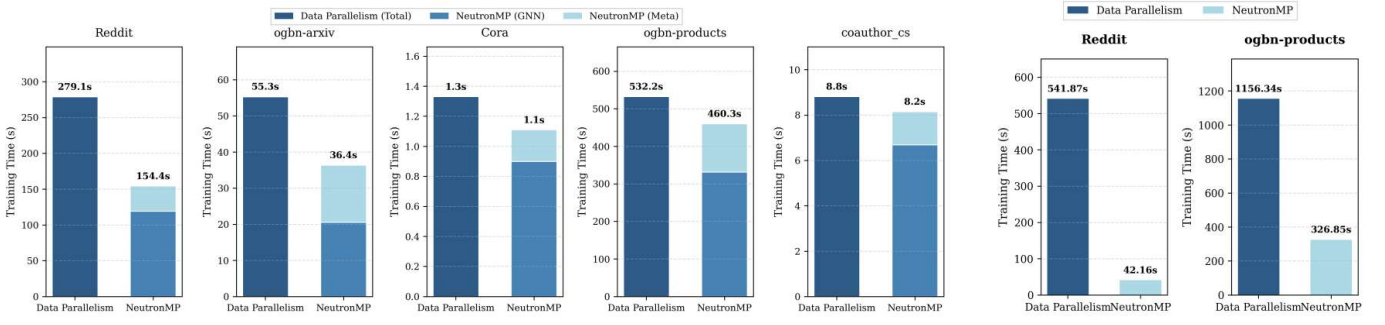


Fig. 5: Training efficiency comparison between DGL-based Data Parallelism and NeutronMP, covering both standard benchmarks and distributed simulations under realistic workload fluctuations.

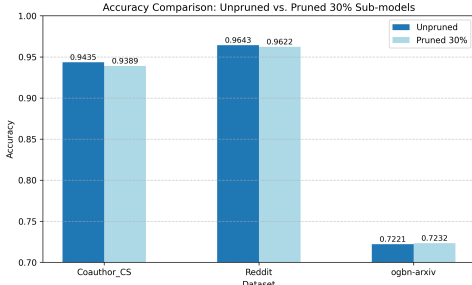


Fig. 6: Accuracy Comparison after Model Pruning

b) Software Stack: All experimental environments are built on the Ubuntu 22.04 LTS operating system. The software stack uniformly employs Python 3.12.9, CUDA 12.4, and PyTorch 2.4.0. For experiments involving baseline methods that rely on synchronous communication, NVIDIA NCCL 2.18.1 is used as the underlying communication backend to ensure fair comparison.

c) Datasets: We selected five representative graph datasets: *Reddit*, *Ogbn-products*, *Cora*, *Ogbn-arxiv*, and *Coauthor-CS*. These datasets cover diverse graph structures, including social networks, e-commerce product associations, and academic citations. With node scales ranging from hundreds of thousands to millions, they adequately validate the system’s generality and performance across different data distributions.

B. Efficiency Experiment

We measured the total training time across five datasets, using DGL’s data-parallel implementation as the baseline. To ensure a fair comparison, and given that NeutronMP introduces an additional meta-learner training phase compared to conventional approaches, this overhead was explicitly included in the timing measurements. As shown in Figure 4, NeutronMP achieves a speedup of 1.07× to 1.8× over the data-parallel strategy. These improvements primarily stem from the elimination of synchronization stalls during training and the adoption of a time-aware load alignment strategy. Furthermore, as a controlled variable, we employed a neighbor

replication technique to mitigate specific synchronization and communication overheads in the data-parallel baseline. This conservative setup substantially diminishes the inherent advantage typically associated with the multi-model parallel scheme. In practical deployment, achieving equivalent final accuracy is typically the sole requirement. When evaluated under a more conventional neighbor-communication strategy by comparing the minimum training time required to reach identical accuracy, NeutronMP achieves up to a 12.85× speedup in extreme cases. While this peak value is partly attributable to the high marginal training cost characteristic of the Reddit dataset, it remains undeniable that NeutronMP delivers substantial advantages in real-world training environments.

We also conducted experiments with **Sancus**, a staleness-aware full-graph framework. However, on the datasets evaluated in this study (ranging from small to medium-large scales), Sancus failed to outperform our optimized DGL data-parallel baseline. This is largely because Sancus’s communication-avoiding mechanisms introduce non-trivial computational and synchronization overheads (e.g., maintaining historical embeddings) that outweigh their benefits on graphs that do not exhibit extreme-scale communication bottlenecks. Consequently, we focused our main comparison on the standard data-parallel strategy, which serves as a more consistent and robust baseline across the diverse dataset sizes used in our evaluation.

C. Robustness Experiment

To validate the inference flexibility of NeutronMP, we simulate a scenario in which one third of the submodels are silenced, for example due to hardware constraints necessitating model pruning. Without retraining the base model, we directly replicate the embedding outputs of the highest accuracy submodel to substitute for the missing submodels’ outputs and perform minor fine tuning of the meta learner’s aggregation function. Experimental results show that this strategy consistently confines accuracy fluctuations to within plus or minus 0.5 percent across all test datasets. On ogbn arxiv, accuracy even improves by 0.1 percent. These findings demonstrate that the overall model performance remains stable despite partial submodel pruning and may even slightly improve due to reduced noise. This highlights NeutronMP’s capability to

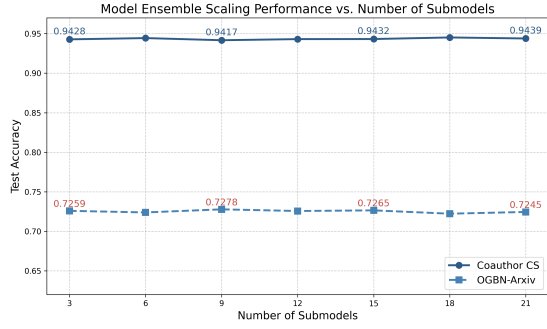


Fig. 7: As the simulation scale increases, the expansion of the embedding data does not impair the integrator’s working capability.

support lightweight dynamic inference architecture adjustments without requiring full retraining, thereby offering high deployment flexibility.

D. Scalability Experiment

Due to the impracticality of leasing a large number of commercial servers for experimentation, our study primarily focuses on simulation-based experiments. For the specific architecture of NeutronMP, the primary concern regarding its scalability is whether the intermediate embeddings will expand multiplicatively under large-scale training, and whether the integrator would suffer from performance degradation when faced with a surge in input data, as well as whether it can continue to effectively integrate information from subgraphs. To address these concerns, we conducted extreme stress tests on two small- to medium-scale datasets, ogbn-arxiv and coauthor-cs. We simulated training scenarios with up to 21 times the demand for intermediate embeddings by employing a strategy that combines original embeddings with noisy embeddings—specifically, replicating the original embeddings and adding moderate noise to evaluate the integrator’s capability to discern useful information under extreme conditions. On both datasets, the method achieved stable final accuracy, with fluctuations remaining within 1% across settings ranging from 3× to 21×.

E. Ablation Study

We compare the performance of MLP and Transformer as meta-learners on three datasets: Coauthor-CS, Reddit, and ogbn-arxiv. As shown in Figure 8, MLP achieves higher accuracy across all datasets—94.26% versus 93.92% on Coauthor-CS, 96.42% versus 96.06% on Reddit, and 72.33% versus 71.67% on ogbn-arxiv—while also requiring significantly less training time. Specifically, the training time of the Transformer is 2.9 times that of MLP on Coauthor-CS, 3.0 times on Reddit, and 3.9 times on ogbn-arxiv, resulting in an average overhead approximately 3.3 times higher. These results indicate that, for the embedding fusion task, the structurally simpler MLP is not only more computationally efficient but also exhibits better generalization. In contrast, the Transformer, due to its

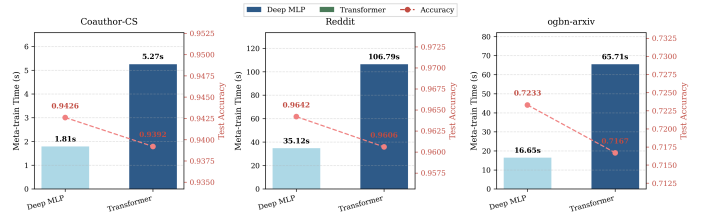


Fig. 8: Meta-Learner Architecture Performance Comparison

larger number of parameters and susceptibility to overfitting, underperforms when meta-training data is limited. Therefore, we adopt MLP as the default meta-learner in NeutronMP.

VI. CONCLUSION

In this paper, we proposed NEUTRONMP, a distributed GNN training framework based on Multi-Model Parallelism to address synchronization bottlenecks in fluctuating cloud environments. By decoupling submodel training and employing a stacking-based ensemble integration, our approach eliminates inter-worker synchronization barriers while maintaining model expressiveness. Theoretical analysis confirms the convergence and generalization capabilities of the paradigm. System optimizations, including I/O-compute role isolation and topology-aware pruning, further enhance efficiency. Experimental results demonstrate that NEUTRONMP significantly reduces training latency and exhibits superior robustness against resource fluctuations compared to state-of-the-art baselines, offering a scalable solution for large-scale graph learning.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (62461146205, U2241212, 625B2041), and the Distinguished Youth Foundation of Liaoning Province (2024021148-JH3/501).

REFERENCES

- [1] R. Duan, C. Yan, J. Wang, and C. Jiang, “Graph ensemble neural network,” *Inf. Fusion*, vol. 110, p. 102461, 2024, doi: 10.1016/j.inffus.2024.102461.
- [2] Q. Lin, S. Yu, K. Sun, W. Zhao, O. Alfarraj, A. Tolba, and F. Xia, “Robust graph neural networks via ensemble learning,” *Mathematics*, vol. 10, no. 8, p. 1300, 2022, doi: 10.3390/math10081300.
- [3] Z. H. Wong, L. Yue, and Q. Yao, “Ensemble learning for graph neural networks,” *arXiv preprint arXiv:2310.14166*, 2023.
- [4] W. Wei, M. Qiao, and D. Jadav, “GNN-ensemble: Towards random decision graph neural networks,” in *Proc. IEEE Int. Conf. Big Data (BigData)*, 2023, pp. 956–965.
- [5] X. Ai, H. Yuan, Z. Ling, Q. Wang, Y. Zhang, Z. Fu, C. Chen, Y. Gu, and G. Yu, “NeutronTP: Load-balanced distributed full-graph GNN training with tensor parallelism,” *Proc. VLDB Endow.*, vol. 18, no. 2, pp. 173–186, 2024, doi: 10.14778/3705829.3705837.
- [6] M. Cao, C. Cao, Y. Zhang, Z. Fu, X. Ai, Q. Wang, Y. Gu, and G. Yu, “NeutronCloud: Resource-aware distributed GNN training in fluctuating cloud environments,” *Proc. VLDB Endow.*, vol. 19, no. 1, pp. 56–69, 2025, doi: 10.14778/3705829.3705837.
- [7] S. Ghadimi and G. Lan, “Stochastic first- and zeroth-order methods for nonconvex stochastic programming,” 2013, *arXiv:1309.5549*. [Online]. Available: <https://arxiv.org/abs/1309.5549>
- [8] A. Maurer, “A vector-contraction inequality for Rademacher complexities,” in *Algorithmic Learning Theory*, Cham: Springer, 2016, pp. 3–17.

- [9] P. L. Bartlett, D. J. Foster, and M. Telgarsky, "Spectrally-normalized margin bounds for neural networks," 2017, arXiv:1706.08498. [Online]. Available: <https://arxiv.org/abs/1706.08498>
- [10] N. Golowich, A. Rakhlin, and O. Shamir, "Size-independent sample complexity of neural networks," in Proc. Conf. Learn. Theory (COLT), PMLR, 2018, pp. 297–299.
- [11] G. Corso, H. Stark, S. Jegelka, et al., "Graph neural networks," Nat. Rev. Methods Primers, vol. 4, no. 1, p. 17, 2024.
- [12] D. Zheng, C. Ma, M. Wang, et al., "DistDGL: Distributed graph neural network training for billion-scale graphs," in Proc. IEEE/ACM 10th Workshop Irreg. Appl.: Archit. Algorithms (IA3), 2020, pp. 36–44.
- [13] V. Md, S. Misra, G. Ma, et al., "DistGNN: Scalable distributed training for large-scale graph neural networks," in Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC), 2021, pp. 1–14.
- [14] J. Vatter, R. Mayer, and H. A. Jacobsen, "The evolution of distributed systems for graph neural networks and their origin in graph processing and deep learning: A survey," ACM Comput. Surv., vol. 56, no. 1, pp. 1–37, 2023.
- [15] S. Wang, Y. Zheng, and X. Jia, "SecGNN: Privacy-preserving graph neural network training and inference as a cloud service," IEEE Trans. Serv. Comput., vol. 16, no. 4, pp. 2923–2938, 2023.
- [16] L. Faber and R. Wattenhofer, "GwAC: GNNs with asynchronous communication," in Proc. Learn. Graphs Conf. (LoG), PMLR, 2024, pp. 8:1–8:20.
- [17] M. Besta and T. Hoefler, "Parallel and distributed graph neural networks: An in-depth concurrency analysis," IEEE Trans. Pattern Anal. Mach. Intell., vol. 46, no. 5, pp. 2584–2606, May 2024.
- [18] Z. Chai, G. Bai, L. Zhao, et al., "Distributed graph neural network training with periodic historical embedding synchronization," 2022, arXiv:2206.00057. [Online]. Available: <https://arxiv.org/abs/2206.00057>
- [19] Z. Li, J. Huang, Y. Li, et al., "Accelerating distributed graph learning by using collaborative in-network multicast and aggregation," in Proc. USENIX Annu. Tech. Conf. (USENIX ATC), 2025, pp. 233–247.
- [20] M. Liu, H. Gao, and S. Ji, "Towards deeper graph neural networks," in Proc. KDD, 2020, pp. 338–348.
- [21] L. Ma, Z. Yang, Y. Miao, et al., "NeuGraph: Parallel deep neural network computation on large graphs," in Proc. USENIX Annu. Tech. Conf. (USENIX ATC), 2019, pp. 443–458.
- [22] H. Peng, J. Li, Y. He, et al., "Large-scale hierarchical text classification with recursively regularized deep Graph-CNN," in Proc. World Wide Web Conf. (WWW), 2018, pp. 1063–1072.
- [23] J. Chen, Z. Chen, and X. Qian, "GNNPipe: Scaling deep GNN training with pipelined model parallelism," 2023, arXiv:2308.10087. [Online]. Available: <https://arxiv.org/abs/2308.10087>
- [24] X. Wan, K. Xu, X. Liao, Y. Jin, K. Chen, and X. Jin, "Scalable and efficient full-graph GNN training for large graphs," Proc. ACM Manag. Data, vol. 1, no. 2, Art. no. 143, 2023.
- [25] Z. Bian, Q. Xu, B. Wang, and Y. You, "Maximizing parallelism in distributed training for huge neural networks," 2021, arXiv:2105.14450. [Online]. Available: <https://arxiv.org/abs/2105.14450>
- [26] B. Wang, Q. Xu, Z. Bian, and Y. You, "2.5-dimensional distributed model training," 2021, arXiv:2105.14500. [Online]. Available: <https://arxiv.org/abs/2105.14500>
- [27] J. Song, H. Jang, H. Lim, et al., "GraNNDIS: Fast distributed graph neural network training framework for multi-server clusters," in Proc. Int. Conf. Parallel Archit. Compil. Tech. (PACT), 2024, pp. 91–107.
- [28] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018.
- [29] Q. Wang, Y. Zhang, H. Wang, et al., "NeutronStar: Distributed GNN training with hybrid dependency management," in Proc. SIGMOD, 2022, pp. 1301–1315.
- [30] Q. Wang, Y. Chen, W.-F. Wong, and B. He, "HongTu: Scalable full-graph GNN training on multiple GPUs (via communication-optimized CPU data offloading)," 2023, arXiv:2311.14898. [Online]. Available: <https://arxiv.org/abs/2311.14898>
- [31] R. Zhu, K. Zhao, H. Yang, et al., "AliGraph: A comprehensive graph neural network platform," Proc. VLDB Endow., vol. 12, no. 12, pp. 2094–2105, 2019.
- [32] J. Zhou, G. Cui, S. Hu, et al., "Graph neural networks: A review of methods and applications," AI Open, vol. 1, pp. 57–81, 2020.
- [33] S. Athlur, N. Saran, M. Sivathanu, R. Ramjee, and N. Kwatra, "Varuna: Scalable, low-cost training of massive deep learning models," in Proc. 17th Eur. Conf. Comput. Syst. (EuroSys), 2022, pp. 472–487.
- [34] Z. Cai, X. Yan, Y. Wu, K. Ma, J. Cheng, and F. Yu, "DGCL: An efficient communication library for distributed GNN training," in Proc. 16th Eur. Conf. Comput. Syst. (EuroSys), 2021, pp. 130–144.
- [35] C. Cao, X. Ai, Q. Wang, et al., "NeutronHeter: Optimizing distributed graph neural network training for heterogeneous clusters," Proc. ACM Manag. Data, vol. 3, no. 4, Art. no. 257, 2025.
- [36] S. Cavallari, V. W. Zheng, H. Cai, K. C.-C. Chang, and E. Cambria, "Learning community embedding with community detection and node embedding on graphs," in Proc. ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2017, pp. 377–386.
- [37] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks," in Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD), 2019, pp. 257–266.
- [38] Alibaba Cloud, "VGPU accelerated instance families - Elastic GPU Service," 2025. [Online]. Available: <https://www.alibabacloud.com/help/en/elastic-gpu-service/latest/vgpu-accelerated-instance-families>
- [39] J. Dean, G. Corrado, R. Monga, et al., "Large scale distributed deep networks," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 25, 2012, pp. 1232–1240.
- [40] J. Du, S. Zhang, G. Wu, J. M. F. Moura, and S. Kar, "Topology adaptive graph convolutional networks," 2017, arXiv:1710.10370. [Online]. Available: <https://arxiv.org/abs/1710.10370>
- [41] Alibaba, "Euler: System introduction," 2019. [Online]. Available: <https://github.com/alibaba/euler/wiki/System-Introduction>
- [42] W. Fan, Y. Ma, Q. Li, et al., "Graph neural networks for social recommendation," in Proc. World Wide Web Conf. (WWW), 2019, pp. 417–426.
- [43] S. Gandhi and A. P. Iyer, "P3: Distributed deep graph learning at scale," in Proc. 15th USENIX Symp. Oper. Syst. Design Implement. (OSDI), 2021, pp. 551–568.
- [44] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 1024–1034.
- [45] A. Harlap, H. Cui, W. Dai, et al., "Addressing the straggler problem for iterative convergent parallel ML," in Proc. 7th ACM Symp. Cloud Comput. (SoCC), 2016, pp. 98–111.
- [46] W. Hu, M. Fey, M. Zitnik, et al., "Open graph benchmark: Datasets for machine learning on graphs," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, 2020.
- [47] K. Huang, J. Zhai, Z. Zheng, Y. Yi, and X. Shen, "Understanding and bridging the gaps in current GNN performance optimizations," in Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program. (PPoPP), 2021, pp. 119–132.
- [48] Z. Jia, S. Lin, M. Gao, M. Zaharia, and A. Aiken, "Improving the accuracy, scalability, and performance of graph neural networks with Roc," in Proc. Mach. Learn. Syst. (MLSys), 2020.
- [49] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," SIAM J. Sci. Comput., vol. 20, no. 1, pp. 359–392, 1998.
- [50] S. Khuller, A. Moss, and J. Naor, "The budgeted maximum coverage problem," Inf. Process. Lett., vol. 70, no. 1, pp. 39–45, 1999.
- [51] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, "Graph convolutional neural networks for web-scale recommender systems," in Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD), 2018, pp. 974–983.
- [52] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. K. Prasanna, "GraphSAINT: Graph sampling based inductive learning method," in Proc. 8th Int. Conf. Learn. Represent. (ICLR), 2020.
- [53] D. Zhang, X. Huang, Z. Liu, et al., "AGL: A scalable system for industrial-purpose graph machine learning," Proc. VLDB Endow., vol. 13, no. 12, pp. 3125–3137, 2020.
- [54] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 31, 2018, pp. 5171–5181.